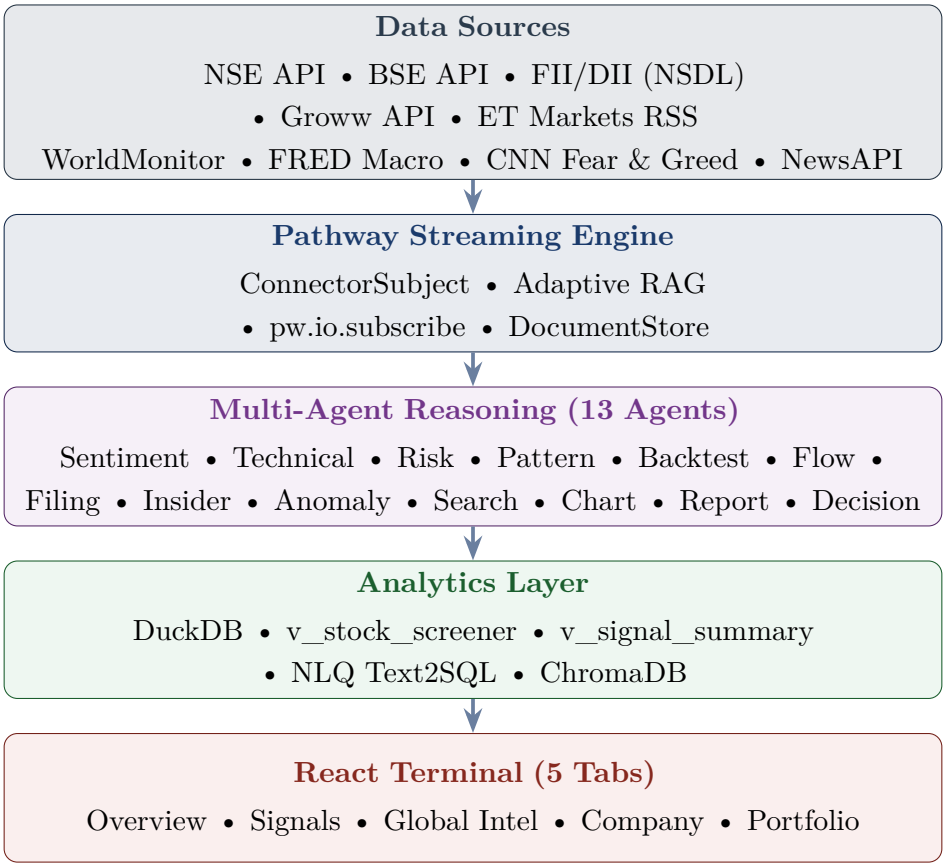
AlphaStream India

Real-Time AI Investment Intelligence
for the Indian Investor



Competition:	ET AI Hackathon 2026
Problem:	PS-6 — AI for the Indian Investor
Repository:	`https://github.com/wildcraft958/AlphaStream_India`
Date:	March 2026

Contents

1 Executive Summary **3**

2 Problem Statement **4**
2.1 The Gap . . . 4
2.2 Our Solution: Live AI . . . 4

3 System Architecture **4**
3.1 Five-Layer Architecture . . . 4
3.2 Data Flow . . . 5

4 Pathway Streaming Integration **6**
4.1 Pathway Features Used . . . 6
4.2 Adaptive RAG — Geometric Retrieval . . . 6
4.3 Fallback Strategy . . . 7

5 Multi-Agent Reasoning System **7**
5.1 Overview . . . 7
5.2 Agent Map . . . 8
5.3 Agent Specifications . . . 9
5.4 Error Handling . . . 9

6 NLQ Agent Pipeline **10**

7 Technology Stack **10**
7.1 Backend . . . 11
7.2 Frontend . . . 11

8 API Reference **11**
8.1 Market Intelligence (15 endpoints) . . . 12
8.2 Global Market (9 endpoints) . . . 12
8.3 Core + Streaming . . . 12

9 Business Impact Model **13**
9.1 Market Sizing (TAM / SAM / SOM) . . . 13
9.2 User Impact . . . 13
9.3 ET Markets Revenue Projection . . . 13
9.4 Competitive Moat . . . 14

10 Setup & Quick Start **14**
10.1 Prerequisites . . . 14

10.2 Step-by-Step 14
10.3 Minimal Mode (No API Keys) 15
10.4 Key Environment Variables 15

11 Demonstration **15**
11.1 Running the Demo Pipeline 15
11.2 Expected Latencies 16

1. Executive Summary

One-Line Pitch

AlphaStream India is a Bloomberg-style AI investment terminal that turns ET Markets data into actionable, real-time signals for India's 14 crore+ retail investors.

AlphaStream India is a production-grade investment intelligence platform built on **Pathway streaming RAG** and a **13-agent AI reasoning pipeline**. It addresses a fundamental gap in the Indian retail investment ecosystem: institutional-grade analytics are locked behind expensive Bloomberg or Reuters terminals, leaving the vast majority of retail investors dependent on WhatsApp tips and delayed news.

Key Differentiators

- **Pathway Adaptive RAG** — news-to-recommendation latency under 2 seconds via incremental streaming (not batch processing).
- **13 specialised AI agents** — each focuses on one analytical dimension (Sentiment, Technical, Risk, FII/DII Flow, Filing, Insider, Anomaly, . . .); the Decision Agent fuses them all into a single Alpha Score.
- **Text2SQL NLQ** — grounded answers from a live DuckDB database, not LLM hallucination. Ask in plain English; get SQL-verified facts.
- **India-first context** — Rs. currency, IST timezone, NSE/BSE bare tickers, FII/DII flows, BSE corporate filings, Groww fundamentals.
- **Bloomberg-style 5-tab terminal** — Overview, Signals, Global Intel, Company, Portfolio — each tab maps to a distinct investor workflow.

Submission Artefacts

GitHub Repository	`https://github.com/wildcraft958/AlphaStream_India`
Architecture Doc	`docs/architecture.md` (this document)
Impact Model	Section 9
Demo Video	`docs/VIDEO_DISCUSSION_SCRIPT.md` (script)

2. Problem Statement

2.1 The Gap

India has **14 crore+ demat accounts** (CDSL + NSDL, 2024) yet the tools available to retail investors are dramatically inferior to those used by institutional traders:

- **Stale data** — most free platforms update prices every 5–15 minutes; institutional desks consume tick data in real time.
- **No signal aggregation** — a retail investor checks RSI manually on one screen, reads FII data on NSDL's PDF portal, and hunts for insider trades on NSE's filing search — three separate workflows with no synthesis.
- **LLM tools are ungrounded** — existing "AI chatbots" for investing answer questions with trained weights, not live market data. They hallucinate stock prices and miss earnings announced an hour ago.
- **2+ hours of daily research** — surveys consistently show active retail investors spend 2–3 hours per day on research that could be automated.

2.2 Our Solution: Live AI

AlphaStream implements the *Live AI* paradigm:

Traditional AI Tool	AlphaStream
Call GPT-4 → text blob	Text2SQL → grounded DuckDB answer
No verification	Historical backtest with win rates
Mock / delayed data	Live NSE / BSE / FII data
Streamlit dashboard	Production React + WebSocket
Single ticker lookup	13-signal Alpha Score fusion
No guardrails	Input/output guardrails + SQL correction loop
Generic (global tickers)	India-first (NSE/BSE, Rs., IST, FII/DII)

3. System Architecture

3.1 Five-Layer Architecture

The system is organised into five vertically-stacked layers. Each layer communicates with the next via well-defined interfaces: REST calls, Pathway streaming callbacks, WebSocket events, and DuckDB SQL.

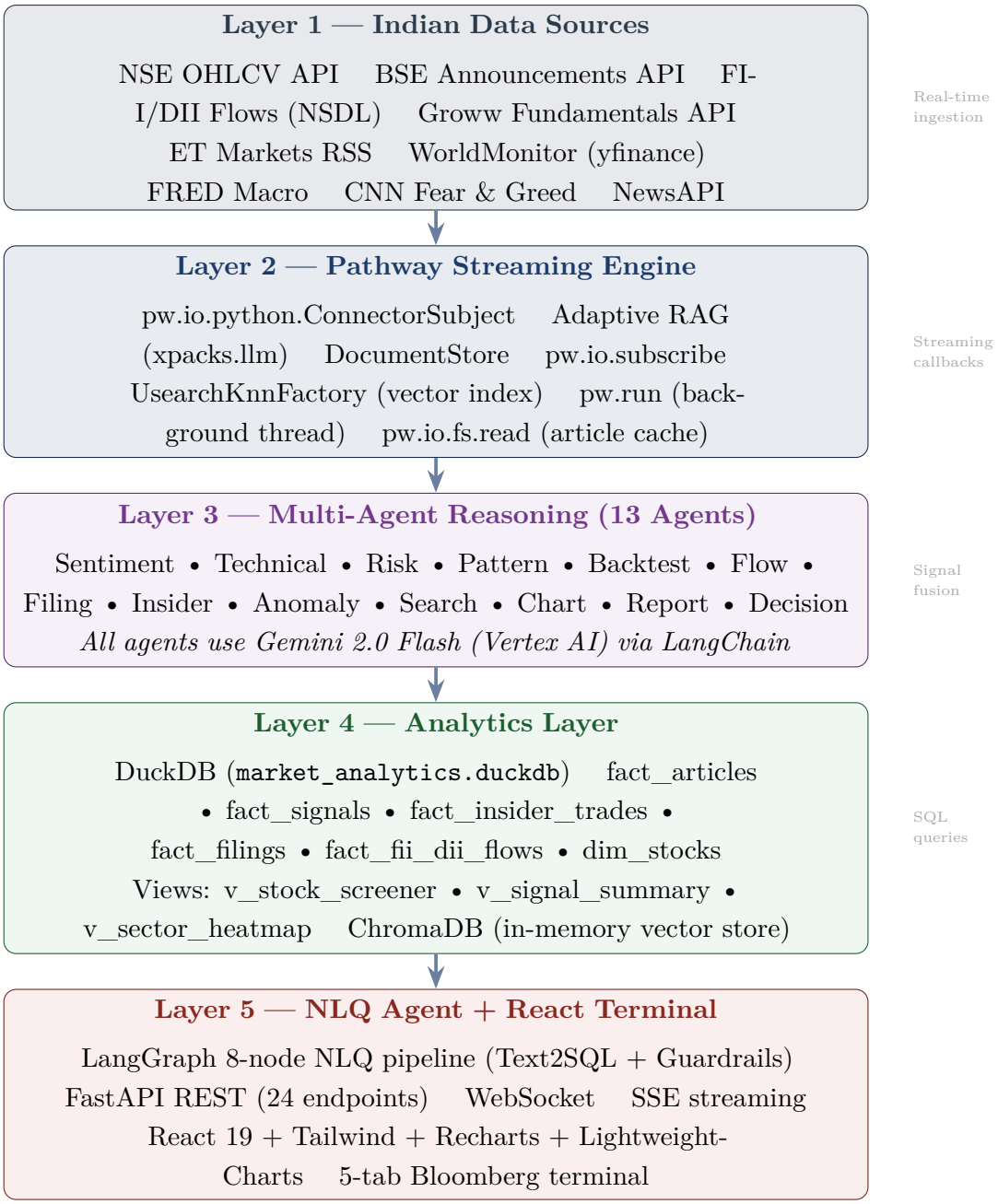


Figure 1: AlphaStream India — Five-Layer Architecture

3.2 Data Flow

1. **Ingest**: The Pathway `ConnectorSubject` polls 5 news APIs in parallel every 60 seconds; articles are written to `data/articles/`.

2. **Index**: `pw.io.fs.read` detects new files and the `DocumentStore` chunks, embeds (all-MiniLM-L6-v2), and indexes them into UsearchKnnFactory — all incrementally, with no full rebuild.

3. **Callback**: `pw.io.subscribe` fires `on_new_article()` on each new row, triggering DuckDB persistence and WebSocket broadcasts.

4. **Recommend**: On a `POST /recommend` request, 12 specialist agents run in parallel (via `asyncio.gather`), and the Decision Agent synthesises their scores into a single BUY/HOLD/SELL recommendation with an Alpha Score.

5. **Deliver**: The response is pushed to the React dashboard over WebSocket (`ws://localhost:8000/ws`) in under 50 ms.

4. Pathway Streaming Integration

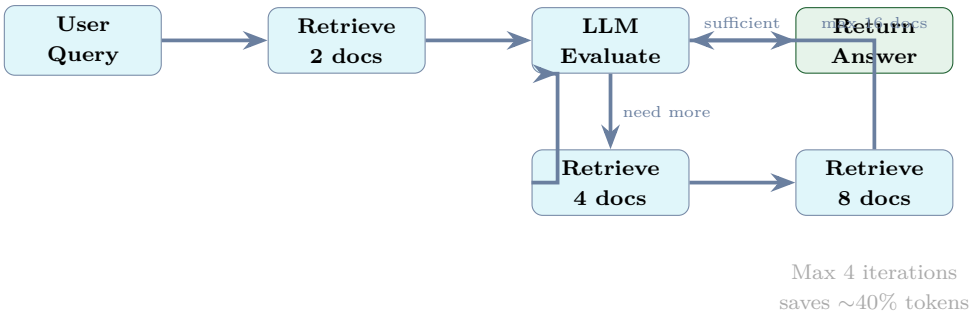
AlphaStream uses Pathway as its **primary real-time engine** — not as a library bolt-on but as the backbone of the entire ingestion and retrieval pipeline.

4.1 Pathway Features Used

Pathway API	File	Purpose
`pw.Schema`	`news_connector.py`	Type-safe streaming schema
`pw.Table`	`pathway_tables.py`	Streaming market data tables
`pw.io.python .ConnectorSubject`	`news_connector.py`	Custom polling connector
`pw.io.fs.read`	`adaptive_rag_server.py`	Auto-detect new articles
`pw.io.subscribe`	`app.py`	Real-time callbacks
`pw.run`	`app.py`	Background engine thread
`pw.apply`	`pathway_tables.py`	UDF transformations
`pw.filter`	`pathway_tables.py`	Event filtering
`pw.reducers`	`pathway_tables.py`	Aggregations (avg, count, max)
`pw.indexing .UsearchKnnFactory`	`adaptive_rag_server.py`	Vector similarity search
`pw.xpacks.llm.*`	`adaptive_rag_server.py`	Official LLM components
`AdaptiveRAG QuestionAnswerer`	`adaptive_rag_server.py`	Geometric retrieval
`pw.persistence`	`pathway_rag.yaml`	Caching & fault tolerance

4.2 Adaptive RAG — Geometric Retrieval

The Adaptive RAG starts with a minimal document set and expands geometrically only when the LLM signals insufficient context:



4.3 Fallback Strategy

When `ENABLE_PATHWAY=false` (default for quick setup), the system uses a synchronous `RAGPipeline` backed by ChromaDB with in-memory embeddings. All API endpoints behave identically — only the latency profile changes.

5. Multi-Agent Reasoning System

5.1 Overview

Every recommendation is produced by **13 specialist agents** running in parallel. Each agent is a LangChain chain with a narrow, well-defined responsibility. The Decision Agent receives all outputs and synthesises a final signal.

5.2 Agent Map

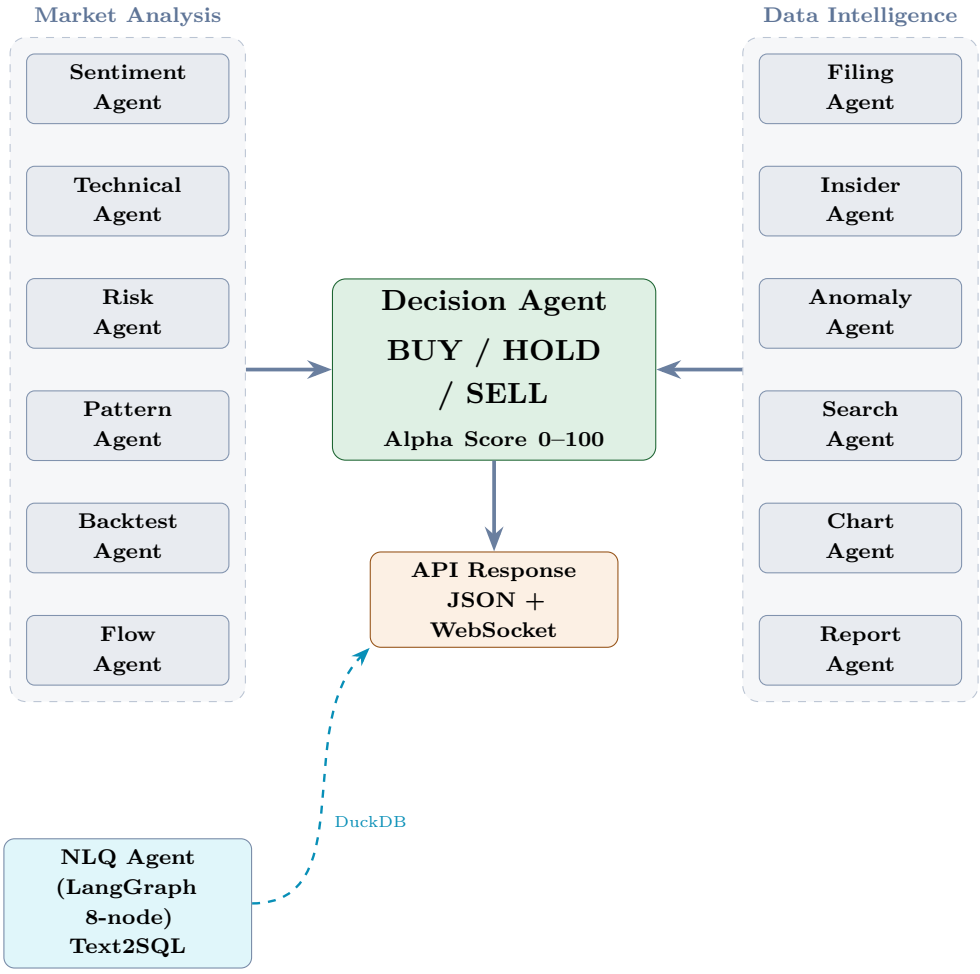


Figure 2: Multi-Agent Reasoning System

5.3 Agent Specifications

Agent	Input	Output	Technology
Sentiment	News articles	Score (−1 to +1), label	LangChain + Gemini
Technical	NSE OHLCV	RSI, SMA trend signals	yfinance + numpy
Risk	Price data	Volatility, stop-loss	Statistical calc
Pattern	OHLCV (3 mo)	Chart patterns detected	Custom detectors
Backtest	Ticker + pattern	Win rate, Sharpe (5yr)	yfinance + stats
Flow	FII/DII data	Net flow signal, streak	NSDL + analysis
Filing	BSE annc.	Materiality, impact	BSE API + Gemini
Insider	NSE SAST/PIT	Cluster buy/sell	NSE conn. + Gemini
Anomaly	OHLCV series	Anomaly flags + scores	River ML (HST)
Search	NLQ query	Reformulated results	LangGraph cyclic
Chart	Ticker + events	PNG chart image	Matplotlib
Report	All outputs	PDF report	ReportLab
Decision	All agents + global	BUY/HOLD/SELL + Alpha	LangChain + Gemini

5.4 Error Handling

Failure Scenario	Handling
Individual agent exception	Caught; score defaults to 0; other agents continue
LLM timeout / API error	Exponential backoff × 3; falls back to rule-based score
Pathway callback error	try/except in `on_new_article`; logs `ERROR`; Pathway thread continues
DuckDB write contention	`threading.Lock` guards all writes to `last_ingestion_time`
Graceful shutdown	`SIGINT/SIGTERM` handler kills Pathway subprocess and waits
Missing GCP credentials	Pydantic `model_validator` warns at startup; `SystemExit` on Vertex AI call

6. NLQ Agent Pipeline

The NLQ (Natural Language Query) system converts free-form investor questions into SQL queries against the live DuckDB analytics database, returning grounded answers.

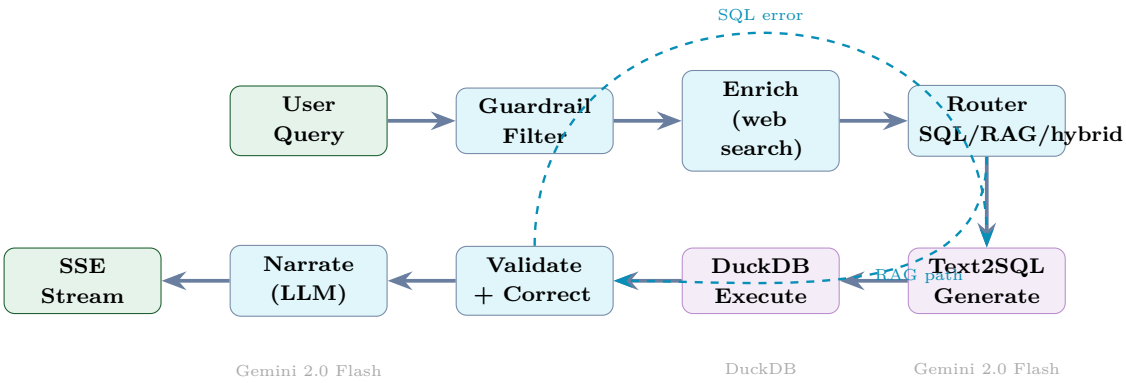


Figure 3: LangGraph 8-Node NLQ Pipeline

#	Node	Responsibility
1	Guardrail	Rejects off-topic / harmful queries; returns canned response
2	Enrich	Optional web search for context not in DB (e.g., recent events)
3	Router	Classifies query as SQL, RAG, or hybrid; picks execution path
4	Text2SQL	Generates SQL against DuckDB schema using Gemini 2.0 Flash
5	Execute	Runs query; captures result set or error
6	Validate	Checks result for empty set, type errors; loops back to SQL on error
7	Narrate	Converts tabular result into natural-language answer with sources
8	SSE Stream	Yields tokens over Server-Sent Events for progressive display

7. Technology Stack

7.1 Backend

Component	Technology	Role
Streaming Engine	Pathway	Real-time RAG, <2 s latency
Web Framework	FastAPI + WebSocket + SSE	REST + real-time push
LLM	Gemini 2.0 Flash (Vertex AI)	All agent reasoning
Agent Framework	LangChain + LangGraph	Multi-agent orchestration
Analytics DB	DuckDB	Pre-aggregated views, screener, Text2SQL
Vector Search	ChromaDB	Embedding-based article retrieval
Anomaly Detection	River ML (HalfSpaceTrees)	Online price/volume anomalies
Indian Data	NSE API, BSE API, Groww, NSDL	OHLCV, filings, FII/DII, PE/PB
Global Data	WorldMonitor (yfinance), FRED	Indices, crypto, FX, macro
Package Manager	uv	Fast, reproducible dependencies
Runtime	Python 3.11	Backend runtime

7.2 Frontend

Component	Technology	Role
Framework	React 19	UI framework
Build Tool	Vite 6	Dev server + bundling
Styling	Tailwind CSS 4	Utility-first CSS
Components	shadcn/ui	Accessible UI primitives
State	Zustand (persisted)	Client state + localStorage
Candlesticks	lightweight-charts 5.1	TradingView-style charting
Data Charts	Recharts 3.6	Bar, Pie, Radar, Line charts
Icons	Lucide React	Consistent iconography

8. API Reference

8.1 Market Intelligence (15 endpoints)

Endpoint	Method	Description
/api/ohlcv/{ticker}	GET	Candlestick + optional RSI/SMA (?indicators=true)
/api/fundamentals/{ticker}	GET	PE, PB, ROE, Div Yield, 52w H/L
/api/radar	GET	Top signals by Alpha Score
/api/screener	GET	Filter Nifty 50 (DuckDB view)
/api/patterns/{ticker}	GET	Chart pattern detection
/api/backtest/{ticker}/{pat}	GET	Historical backtest (5-year)
/api/flows	GET	FII/DII flows + streak detection
/api/anomalies/{ticker}	GET	River ML anomaly scores
/api/filings/{ticker}	GET	BSE corporate announcements
/api/tickers	GET	Nifty 50 universe
/api/news	GET	Recent articles from DuckDB
/api/portfolio	POST	Set user portfolio holdings
/api/portfolio/summary	GET	Live P&L summary
/api/insights	GET	Ambient AI alerts
/ingest	POST	Manual article injection

8.2 Global Market (9 endpoints)

Endpoint	Method	Description
/api/global/indices	GET	NIFTY, SENSEX, S&P 500, DOW ...
/api/global/commodities	GET	Gold, Crude, Silver, Copper ...
/api/global/crypto	GET	BTC, ETH, SOL, XRP
/api/global/currencies	GET	INR/USD, DXY
/api/global/sectors	GET	12 US sector ETF returns
/api/global/fear-greed	GET	CNN Fear & Greed index
/api/global/macro	GET	FRED macro signals + verdict
/api/global/vix	GET	VIX volatility index
/api/global/geo-risk	GET	India geopolitical risk score

8.3 Core + Streaming

Endpoint	Method	Description
/recommend	POST	Full 13-agent recommendation
/api/nlq	POST	NLQ blocking response
/api/nlq/stream	POST/GET	NLQ with SSE streaming
/insider/{ticker}	GET	NSE SAST/PIT insider trading
/report/{ticker}	POST	PDF report generation
/ws/stream/{ticker}	WS	Real-time rec + market updates

9. Business Impact Model

9.1 Market Sizing (TAM / SAM / SOM)

Level	Segment	Size	Rationale
TAM	Total Indian equity market participants	14 Cr	CDSL + NSDL demat accounts (2024)
SAM	Active traders who research daily	∼3 Cr	≈20% of accounts trade monthly
SOM	Realistic 5-year premium capture	1.5 L	0.5% of SAM (conservative freemium rate)

9.2 User Impact

Time Saved Per User

1.75 hours/day × Rs. 500/hr × 250 trading days = **Rs. 2.19 Lakh/user/year** in recovered time.

Signal Type	Method	Win Rate	Avg 30d Return
RSI Divergence	Automated scan	60–78%	+3.26%
MACD Crossover	Real-time detect.	45–57%	+0.71%
Insider Cluster Buy	3+ insiders/30d	High (historical)	Varies
FII Buying Streak	5+ sessions	∼72%	+3–5%

Conservative alpha estimate: 2–3 actionable signals/month × 60% accuracy × Rs. 3,000 avg = **Rs. 54,000/user/year**.

9.3 ET Markets Revenue Projection

	Year 1	Year 3	Year 5
Premium users (Rs. 999/month)	50,000	2,00,000	5,00,000
Annual Revenue	**Rs. 59.9 Cr**	**Rs. 239.8 Cr**	**Rs. 599.4 Cr**
Signal content cost saving	Rs. 5–10 Cr/yr	—	—

Assumptions: Price held constant at Rs. 999/month (upside not modelled). Year 3 growth via portfolio-alert virality. Year 5 via Tier 2/3 city expansion.

9.4 Competitive Moat

1. **India-specific regulatory data** — NSE SAST/PIT and NSDL FII/DII feeds not available through generic global APIs.
2. **Backtested signals** — every signal carries a win rate grounded in NSE/BSE historical data; LLM-only tools cannot replicate this.
3. **Streaming architecture** — Pathway incremental indexing vs. static dashboards that recompute from scratch on every load.
4. **Portfolio-aware alerts** — signals are contextualised to each user's actual holdings.
5. **Compound improvement** — each interaction improves signal ranking; accuracy scales with user base.

10. Setup & Quick Start

10.1 Prerequisites

- Python 3.11+ and `uv` package manager
- Node.js 18+ (frontend build)
- Google Cloud account with Vertex AI enabled (Gemini 2.0 Flash)
- `NEWS_API_KEY` from `newsapi.org` (required for live news)
- 4 GB+ RAM recommended

10.2 Step-by-Step

```
# 1. Clone
git clone https://github.com/wildcraft958/AlphaStream_India.git
cd AlphaStream_India

# 2. Backend dependencies
cd backend
uv sync

# 3. Configure API keys
cp .env.example .env
# Edit .env: set GOOGLE_APPLICATION_CREDENTIALS, GCP_PROJECT_ID,
   NEWS_API_KEY

# 4. Initialise DuckDB with Nifty 50 data
uv run python -m src.data.market_schema

# 5. Start backend (cold start ~75 s due to LangChain imports)
./start.sh          # OR: uv run uvicorn src.api.app:app --port 8000
```

```
# 6. Frontend (new terminal)
cd ../frontend
npm install && npm run dev

# 7. Open dashboard
# http://localhost:5173
```

10.3 Minimal Mode (No API Keys)

Set in `backend/.env`:

```
ENABLE_PATHWAY=false
NEWS_API_KEY=
GCP_PROJECT_ID=
```

The UI loads with cached article data and pre-computed recommendations — sufficient to evaluate the dashboard, agent outputs, and PDF report generation.

10.4 Key Environment Variables

Variable	Purpose
`GOOGLE_APPLICATION_CREDENTIALS`	Path to GCP service-account JSON
`GCP_PROJECT_ID`	GCP project ID (Vertex AI)
`GCP_REGION`	Vertex AI region (default: us-central1)
`NEWS_API_KEY`	NewsAPI key for live article ingestion
`ENABLE_PATHWAY`	`true` = Pathway RAG, `false` = sync fallback
`GROWW_API_TOKEN`	Groww fundamentals (PE, PB, ROE)
`FRED_API_KEY`	FRED macro signals (CPI, yield curve)

11. Demonstration

11.1 Running the Demo Pipeline

```
# Full automated demo (8 steps including NLQ, Radar, FII/DII)
cd AlphaStream_India
python demo_pipeline.py --ticker RELIANCE

# Inject a scenario (bullish or bearish) for a specific stock
python scripts/run_demo.py --ticker RELIANCE --scenario bullish

# Inject a single custom article
cd backend
uv run python scripts/inject_article.py \
  "RELIANCE Buyback Announced" \
```

```
  "Reliance Industries board approves Rs 5000 Cr buyback..." \
  --ticker RELIANCE --source "ET Markets"
```

11.2 Expected Latencies

Operation	Latency
Article ingestion to DuckDB	<100 ms
Pathway RAG indexing	<200 ms
13-agent recommendation	∼1.2 s (LLM-bound)
WebSocket delivery to browser	<50 ms
Total: news → UI update	**<2 seconds**
PDF report generation	∼15 s

Appendix: Repository Structure

```
AlphaStream_India/
+-- backend/
|   +-- src/
|   |   +-- agents/         # 13 AI agents (sentiment, technical, risk,
   ...)
|   |   +-- connectors/     # NSE, BSE, Groww, WorldMonitor, FII/DII
|   |   +-- pipeline/       # Pathway streaming + Adaptive RAG
|   |   +-- api/            # FastAPI application (app.py)
|   |   +-- data/           # DuckDB schema + Nifty 50 seed data
|   |   \-- config.py       # Pydantic settings with GCP validation
|   +-- scripts/            # inject_article.py, demo helpers
|   +-- market_analytics.duckdb
|   \-- start.sh            # One-command startup (respects
   ENABLE_PATHWAY)
+-- frontend/
|   \-- src/
|       +-- components/trading/   # 25+ Bloomberg terminal components
|       +-- pages/Dashboard.tsx   # 5-tab layout
|       +-- services/api.ts       # Typed API client (60+ endpoints)
|       \-- store/appStore.ts     # Zustand state
+-- docs/
|   +-- architecture.md           # Mermaid data-flow diagram
|   +-- impact_model.md           # TAM/SAM/SOM + revenue projections
|   +-- VIDEO_DISCUSSION_SCRIPT.md
|   \-- submission.tex            # This document
+-- demo_pipeline.py              # 8-step automated demo
+-- scripts/
|   +-- run_demo.py               # Scenario injection (bullish/bearish)
|   \-- demo_data.json            # India-focused demo articles
\-- README.md
```